

NDCS Internship Work Summary: 3D Web Viewer Architecture

This document outlines the web development work completed during the NDCS internship period.

The web viewer work was divided mainly into two areas. Nyunt focused on 2D design features in the web application, while Shafik focused on the 3D design viewer and 3D rendering workflow.

This document describes the current 3D surface architecture, including jaw mesh rendering, polyline rendering, artificial teeth rendering, viewer controls, and loading optimisation.

3D Surface Architecture Overview

The 3D viewer is built using Three.js. The main 3D rendering logic is handled in `src/index.js`, while artificial tooth rendering is handled separately in `src/artificialTeeth.js`.

At a high level, the viewer contains the jaw meshes, design polylines, artificial teeth overlays, lights, camera, and user controls.

THREE.Scene

```
├─ parentObject
│   ├── upper/lower jaw mesh objects
│   └── loaded STL/OFF mesh variants
├─ polylineOverlayGroup
│   ├── upper/lower component polyline groups
│   │   ├── tube geometry group
│   │   └── edit handle group
├─ artificial-tooth-overlay-group
│   ├── upper-artificial-teeth group
│   └── lower-artificial-teeth group
├─ lights
└─ camera
```

Core Scene Components

The main scene components are:

- scene: root Three.js scene.
- camera: orthographic camera used to view the 3D model.

- `parentObject`: contains the real jaw meshes and loaded STL/OFF mesh variants.
- `polylineOverlayGroup`: contains the design/component polylines and draggable edit points.
- `artificialTeethRenderer`: module-owned artificial teeth renderer created from `src/artificialTeeth.js`.
- `renderer`: `THREE.WebGLRenderer` attached to `#container3D`.
- `TrackballControls` and `OrbitControls`: used for camera movement, rotation, zoom, and pan behaviour.

Jaw Mesh Pipeline

Jaw meshes are loaded from backend API endpoints in `src/index.js`.

The main endpoints are:

```
/parameterisation/mesh/getall
/surface/getall
/stl/get
```

The viewer supports both OFF and STL mesh loading:

- `src/OFFLoader.js` parses OFF data into `BufferGeometry`.
- `src/STLMeshLoader.js` parses STL data, merges vertices, and creates mesh objects.

The main jaw mesh material uses `MeshPhongMaterial` with vertex colours. Surface or denture meshes use `MeshStandardMaterial`.

All jaw meshes are forced to render as opaque through `enforceOpaqueJawMesh()` so that transparency settings do not accidentally affect the main anatomical surface.

One important transform detail is that some upper OFF meshes are rotated and shifted after loading:

```
changeMeshRotation(mesh, 1, 1, 180);
mesh.position.y += 5;
```

This legacy upper-jaw transform affects how overlays align with the jaw. Because of this, overlay data may be interpreted either in jaw-local coordinates or scene-world coordinates depending on the source.

Polyline System

Polyline rendering logic lives in `src/index.js`.

The polyline data flow is:

```
/polylines/getall  
→ normalizePolylineResponse()  
→ extractPolylineSegments()  
→ renderPolylineData()  
→ createPolylineObjects()  
→ polylineOverlayGroup
```

Each polyline component group contains:

- `positionAttribute`: source point buffer.
- `polyline-line`: rendered tube segments.
- `polyline-edit-points`: draggable sphere handles.
- `userData.component`: component name such as Retainer, Minor Conn, Gingival Points, etc.
- `userData.coordinateSpace`: detected coordinate space, either jaw-local or scene-world.

Polyline coordinate-space detection is handled inside the polyline renderer in `src/index.js`.

The current behaviour is:

1. Compare each polyline segment against jaw vertices in local space and world space.
2. If the segment is jaw-local, copy the jaw mesh transform onto the polyline group.
3. If the segment is already scene-world, leave the polyline group identity unchanged to avoid double-transforming.
4. Minor connector components can be seated against the jaw surface using the correct coordinate space.

A key design decision is that polyline coordinate-space detection belongs in the polyline renderer, not in artificial teeth rendering. This keeps the polyline system independent and prevents artificial tooth logic from interfering with existing polyline behaviour.

Artificial Teeth System

Artificial teeth are handled separately in `src/artificialTeeth.js`.

Artificial teeth are rendered as a separate Three.js overlay layer. They do not modify the jaw mesh or the polyline system.

The current artificial tooth data flow is:

```
/toothPlacementData/get  
→ decode MessagePack / StageDataManager  
→ map toothCraftDataUpperJaw_ / toothCraftDataLowerJaw_  
→ choose geometry source  
→ choose matching placement source  
→ renderData()
```

Artificial teeth are rendered inside:

```
artificial-tooth-overlay-group  
├─ upper-artificial-teeth group  
└─ lower-artificial-teeth group
```

Each artificial tooth is rendered as a 3D mesh object rather than a line or point object. The tooth mesh has its own geometry, material, position, rotation, and transform.

Artificial Tooth Geometry and Placement Rules

The current rendering rule is that the geometry source and placement source must match.

The placement rule is:

```
surfaceMesh geometry -> apiMeshCenter position  
toothData geometry   -> toothData_[7]/[10] position
```

This rule is important because the artificial tooth API can provide different geometry and placement data sources. If geometry from one coordinate space is combined with placement from another coordinate space, teeth can appear far away from the correct jaw location.

When valid API surface mesh geometry exists, the surface mesh is treated as the source of truth. The viewer recentres the mesh geometry around its own API mesh centre, then places the tooth back at that API mesh centre.

When toothData geometry is the active geometry source, the viewer can use toothData placement fields [7] and [10].

Fields [9] and [12] are not used as tooth positions.

The cleaned-up rule is:

Do not mix surfaceMesh geometry with toothData placement.

Artificial Teeth and Guidelines

Buccal and gingival guideline curves are visual aids only.

They do not:

- place teeth
- snap teeth
- constrain teeth
- scale teeth
- orient teeth
- act as fallback placement paths

The current visual convention is:

Buccal guideline -> orange

Gingival guideline -> blue

Artificial teeth do not currently consume rendered polyline paths as guide input. This keeps the artificial tooth system separate from the working polyline system.

Removed Experimental Behaviour

Earlier experimental logic has been removed or disabled to keep artificial tooth rendering faithful to the API data.

The current artificial tooth renderer no longer relies on:

optional guideline placement

snapTeethToJawSurface()

procedural fallback tooth meshes

targeted 180-degree flipping

downward tilt correction

surface-clearance nudging

landmark fallback placement

reference-mode switching

vertex debug overlays

This means artificial teeth now follow the API mesh and placement data more directly.

Viewer Controls

Viewer controls are managed mainly through `src/newControls.js`.

Relevant controls include:

- jaw mesh visibility
- jaw mesh transparency
- undercut and occlusion modes
- polyline visibility controls
- artificial teeth visibility controls

Some controls call globals registered by `src/index.js` and `src/artificialTeeth.js`, such as:

```
window.setPolylineJawVisibility  
window.setArtificialTeethJawVisibility
```

The artificial teeth renderer exposes jaw visibility controls so upper and lower artificial teeth can be shown or hidden independently.

Render Loop

The main render loop is in `src/index.js`.

The current render loop follows this structure:

```
function animate() {  
  requestAnimationFrame(animate);  
  controls.update();  
  anchorViewerRotationTarget(controls);  
  clampViewerControlTarget(controls);  
  artificialTeethRenderer.syncToJawMeshes();  
  renderer.render(scene, camera);  
}
```

The rotation target anchoring and clamp logic help prevent the jaw from drifting out of view while rotating.

`artificialTeethRenderer.syncToJawMeshes()` keeps the artificial teeth overlay groups aligned with their corresponding jaw meshes during interaction.

Loading and Performance Optimisation

The current performance strategy is to make the viewer usable as early as possible.

The loading model is:

1. Load case data.
2. Load jaw mesh data.
3. Render the jaw mesh and start the viewer.
4. Load design overlays such as polylines and artificial teeth in the background.

This keeps the viewer interactive while heavier overlay data continues loading.

The normal console output has been simplified to high-level breadcrumbs such as:

165

Case Info

2D image

Heatmap upper

Heatmap lower

Jaw mesh

Denture mesh

Jaw mesh

Polyline

Artificial Teeth

Detailed timing diagnostics are still available internally through:

`window.viewerLoadTimings`

Manual artificial teeth and polyline audits are also still available when needed, but they do not run noisily during normal loading.

Reliable Loading and UI Optimisation Plan

The biggest current loading bottleneck is large network payload size. Some API responses are between 16 MB and 30 MB, and one large request can take over one minute.

The safe client-side optimisation approach is:

- keep progressive loading
- render jaw meshes first
- load polylines and artificial teeth in the background
- avoid blocking the viewer on non-critical overlays

- cache case-scoped responses during the session
- avoid duplicate login/session calls per API request
- keep the console clean during normal use
- show staged loading feedback in the UI

Suggested UI loading stages:

Loading case

Loading jaw

Viewer ready

Loading design overlays

Artificial teeth ready

Backend improvements that would provide the largest future gains include:

- enabling gzip or Brotli compression
- avoiding base64-in-JSON for large mesh payloads
- delivering binary mesh payloads directly
- splitting large endpoints by purpose
- adding metadata endpoints for cache validation
- supporting partial or lazy asset loading

Mental Model

The clean architecture is:

Jaw mesh = anatomical surface source of truth

Polylines = design/component overlay layer

Artificial teeth = separate prosthetic mesh overlay layer

Guidelines = visual-only reference curves

Controls = visibility, interaction, and editing affordances

The most important rule for artificial teeth is:

Geometry source and placement source must match.

This is what keeps artificial teeth aligned correctly with the jaw while avoiding viewer-side guessing or fallback placement.

NDCS Internship Work Summary: 3D Web Viewer Architecture (Claude)

This document outlines the web development work completed during the NDCS internship period.

The web viewer work was divided mainly into two areas. Nyunt focused on 2D design features in the web application, while Shafik focused on the 3D design viewer and 3D rendering workflow.

This document describes the current 3D surface architecture, including jaw mesh rendering, polyline rendering, artificial teeth rendering, viewer controls, loading optimisation, and the broader application structure.

Application Overview

The application is a multi-page web app served as static files via nginx inside a Docker container. It is not a single-page app — each view is a separate HTML page.

The main pages are:

Page	File	Purpose
Login	<code>index.html</code>	Authentication entry point
Case List	<code>src/pages/case_list.html</code>	Browse, manage, and open cases
3D Viewer	<code>src/pages/ThreeDViewer.html</code>	3D rendering of jaw and design overlays
2D Annotation	<code>src/pages/2DAnnotation.html</code>	2D image markup and annotation

Navigation between pages happens via URL redirects. The 3D viewer receives the selected case via an encrypted `?id=` query parameter in the URL.

The frontend is bundled by webpack. The main entry point is `src/index.js`, which produces `dist/bundle.js`. All other modules are statically imported into the bundle except `src/js/chat.js`, which is lazy-loaded on demand to keep it off the critical module graph.

nginx is configured to serve gzip-compressed JS/CSS responses and apply long-lived cache headers for assets.

Module Structure

```
src/
  index.js           - main viewer entry point, scene setup, mesh/polyline loading
  artificialTeeth.js - artificial teeth renderer (self-contained module)
  newControls.js     - visibility, transparency, and interaction controls
  resetButton.js     - heatmap legend panel and reset button
  ApiClient.js       - authenticated HTTP client with streaming progress
  OFFLoader.js       - OFF mesh format parser → THREE.BufferGeometry
  STLMeshLoader.js   - STL mesh parser → merged THREE.Mesh
  crypt.js           - symmetric XOR case ID encryption/decryption
  control.js         - miscellaneous control helpers
  js/
    viewerShell.js   - viewer page footer, chat wiring, sidebar
```

caseManagement.js	– case list page logic and table population
chat.js	– live chat / notes panel (lazy-loaded)
login.js	– login and OTP flow
2Dannotation.js	– 2D canvas annotation tools
appSidebar.js	– collapsible app sidebar component
toast.js	– non-blocking notification toasts
confirmModal.js	– confirmation dialogs
authGuard.js	– redirect to login if session is missing
connectivityIndicator.js	– online/offline status indicator
notifications.js	– in-app notification panel
versionHistory.js	– case version history panel
AnnotationHistory.js	– annotation history panel
ThreeDMobile.js	– mobile-specific viewer adjustments

Authentication and Session Model

All API calls go through **ApiClient** (`src/ApiClient.js`). Before every request, **ApiClient.post** calls **getSessionLogin()**, which performs a single shared login for the session.

getSessionLogin uses a singleton promise so that multiple concurrent API calls at startup do not each trigger a separate login:

```
let sessionLoginPromise = null;

function getSessionLogin() {
  if (!sessionLoginPromise) {
    sessionLoginPromise = login().catch((error) => {
      sessionLoginPromise = null; // allow retry on failure
      throw error;
    });
  }
  return sessionLoginPromise;
}
```

The login call posts credentials to **/user/login**. On success the server establishes a session. After the viewer finishes loading, a **/user/logout** call is issued in the background to close the shared session.

The viewer uses a fallback service account (**uuid**: **AC4gRQXZJoNz9EhhW36Q8jMjXBsf**) so that shared viewer URLs can load without requiring the viewer user to be authenticated. When a logged-in user is present in **localStorage**, their UUID is preferred.

Case ID Encryption

Case IDs in URLs are obfuscated using a symmetric XOR cipher implemented in **src/crypt.js**. The function **lol()** automatically encrypts or decrypts based on input type:

- Integer input → encrypted URL-safe base64 string

- String input → decrypted integer case ID

The encryption pipeline is: base64-encode the integer → add noise prefix/suffix → XOR with a shared key → URL-safe base64 encode.

This means the same function is used to both produce the `?id=` URL parameter and to decode it on the viewer page. It is not cryptographically secure — its purpose is URL obfuscation, not access control.

3D Surface Architecture Overview

The 3D viewer is built using Three.js. The main 3D rendering logic is handled in `src/index.js`, while artificial tooth rendering is handled separately in `src/artificialTeeth.js`.

At a high level, the viewer contains the jaw meshes, design polylines, artificial teeth overlays, lights, camera, and user controls.

THREE.Scene

- └─ `parentObject`
 - └─ upper/lower jaw mesh objects
 - └─ loaded STL/OFF mesh variants
- └─ `polylineOverlayGroup`
 - └─ upper/lower component polyline groups
 - └─ tube geometry group
 - └─ edit handle group
- └─ `artificial-tooth-overlay-group`
 - └─ upper-artificial-teeth group
 - └─ lower-artificial-teeth group
- └─ `lights`
- └─ `camera`

Camera

The viewer uses a `THREE.OrthographicCamera`. Orthographic projection is used so that the jaw and design overlay appear at a consistent scale regardless of zoom level — this makes it easier to inspect relative proportions of design components without perspective distortion.

Both `TrackballControls` and `OrbitControls` are imported. `TrackballControls` is the active control for rotation and zoom interaction. `OrbitControls` is available as a fallback.

Core Scene Components

Symbol	Role
<code>scene</code>	Root Three.js scene
<code>camera</code>	Orthographic camera
<code>parentObject</code>	Container for jaw mesh objects and loaded variants
<code>polylineOverlayGroup</code>	Container for design polylines and draggable edit handles
<code>artificialTeethRenderer</code>	Self-contained module instance from <code>src/artificialTeeth.js</code>
<code>renderer</code>	<code>THREE.WebGLRenderer</code> attached to <code>#container3D</code>

ApiClient and Streaming Progress

`ApiClient.post` handles all data requests. It reads the response body as a `ReadableStream` to track download progress and update the loading bar in real time. This requires a `Content-Length` response header to calculate percentage.

A known issue: the GCP proxy in front of the API (via: 1.1 google) sometimes strips `Content-Length` or uses chunked transfer encoding. When that happens the client falls back to `response.json()` directly with no streaming progress. Data returned is identical — the loading bar just does not show a percentage for that specific request.

The `ApiClient` also returns the string `'stl'` rather than throwing when the server returns a `500` or `404`. This convention is used internally to signal that a mesh variant does not exist (for example, checking whether a `closed.off` variant is available).

Jaw Mesh Pipeline

Jaw meshes are loaded from backend API endpoints in `src/index.js`. The two main mesh endpoints are fetched in parallel before either is awaited, so the denture mesh download begins while the jaw mesh is being processed:

```
const meshPromises = {
  "/parameterisation/mesh/getall": apiClient.post(..., "Jaw mesh"),
  "/surface/getall":               apiClient.post(..., "Denture mesh"),
};
```

Endpoint	Mesh type
<code>/parameterisation/mesh/getall</code>	Parameterised jaw mesh (OFF format)
<code>/surface/getall</code>	Denture/surface mesh (OFF format)
<code>/stl/get</code>	Closed jaw mesh variant (STL format, availability-checked first)

Format parsers:

- `src/OFFLoader.js` — parses the OFF text format into `THREE.BufferGeometry` with vertex positions and colours.
- `src/STLMeshLoader.js` — parses binary or ASCII STL data, merges duplicate vertices, and creates a `THREE.Mesh`.

Materials:

- Main jaw mesh: `MeshPhongMaterial` with vertex colours.
- Surface/denture mesh: `MeshStandardMaterial`.

All jaw meshes are forced to render as fully opaque through `enforceOpaqueJawMesh()` so that transparency controls applied by the user do not accidentally affect the main anatomical surface.

Upper Jaw Transform

One important legacy detail: upper OFF meshes receive a post-load rotation and Y-position shift:

```
changeMeshRotation(mesh, 1, 1, 180);
mesh.position.y += 5;
```

This transform is applied after loading. It affects how all overlay data aligns with the upper jaw. Overlay data may be in jaw-local space (relative to the pre-transform mesh) or scene-world space (already accounting for the transform) depending on which API produced it. The coordinate-space detection in the polyline renderer handles this distinction — see below.

Heatmap (Undercut/Occlusion)

Heatmap data is fetched from `/undercutheatmap/get` in two calls — one for upper jaw (`jaw_type: 2`) and one for lower jaw (`jaw_type: 1`). These calls are issued concurrently with the main mesh calls.

The heatmap legend panel is managed by `src/resetButton.js`. The panel heading is labelled **Heatmap** and is toggled by the Heatmap button in the controls drawer.

Polyline System

Polyline rendering logic lives in `src/index.js`.

Data Flow

```
/polylines/getall
→ normalizePolylineResponse()
→ extractPolylineSegments()
→ renderPolylineData()
→ createPolylineObjects()
→ polylineOverlayGroup
```

Each polyline component group contains:

Property	Description
<code>positionAttribute</code>	Source point buffer
<code>polyline-line</code>	Rendered tube segment geometry
<code>polyline-edit-points</code>	Draggable sphere handles for editing
<code>userData.component</code>	Component name: Retainer, Minor Conn, Gingival Points, etc.
<code>userData.coordinateSpace</code>	Detected space: <code>jaw-local</code> or <code>scene-world</code>

Coordinate Space Detection

Polyline data from the API can be in either jaw-local coordinates or scene-world coordinates. The polyline renderer detects which space each segment belongs to by comparing segment positions against jaw vertices in

both coordinate spaces. If the segment is jaw-local, the jaw mesh transform is copied onto the polyline group. If it is already in scene-world space, the group transform is left as identity to avoid double-transforming.

This detection belongs in the polyline renderer and not in the artificial teeth system. This keeps the two overlay systems independent.

Polyline Visibility Controls

The polyline panel in the controls drawer allows per-component visibility to be toggled via checkboxes. An eye icon button toggles all components at once. Individual components are identified by colour swatch. A reset button reverts any edits to polyline positions.

Artificial Teeth System

Artificial teeth are handled as a self-contained renderer in `src/artificialTeeth.js`. They form a separate overlay layer and do not modify the jaw mesh or polyline system.

Data Flow

```
/toothPlacementData/get
→ toUint8Array() – normalise raw bytes / base64 / array input
→ decodeMessagePack() – custom inline MessagePack decoder (no external library)
→ StageDataManager – manages per-stage tooth placement records
→ map toothCraftDataUpperJaw_ / toothCraftDataLowerJaw_
→ choose geometry source (surfaceMesh or toothData)
→ choose matching placement source
→ renderData()
→ artificial-tooth-overlay-group
```

The MessagePack decoder is implemented inline in `artificialTeeth.js` without an external library dependency, handling the binary framing format used by the tooth placement API.

Scene Structure

```
artificial-tooth-overlay-group
├─ upper-artificial-teeth group
└─ lower-artificial-teeth group
```

Each artificial tooth is a full 3D mesh object with its own geometry, material, position, rotation, and transform. This is distinct from the polyline system which renders line/tube geometry.

Geometry and Placement Rules

The fundamental rule: **geometry source and placement source must match.**

Geometry source	Placement source
surfaceMesh geometry	apiMeshCenter position

<code>toothData</code> geometry	<code>toothData[7]</code> or <code>toothData[10]</code> position
---------------------------------	--

When valid surface mesh geometry exists from the API, the mesh is recentred around its own API mesh centre and placed back at that centre. This prevents mixing coordinate spaces between geometry and placement.

`toothData` fields `[9]` and `[12]` are not used as tooth position sources.

Guidelines

Buccal and gingival guideline curves are visual reference aids rendered as coloured lines:

Guideline	Colour
Buccal	Orange
Gingival	Blue

Guidelines do not place, snap, constrain, scale, or orient teeth. They do not act as fallback placement paths. Artificial teeth do not consume polyline paths as guide input.

Removed Experimental Behaviour

Earlier experimental logic has been removed to keep artificial tooth rendering faithful to the API data. The current renderer no longer relies on: optional guideline placement, `snapTeethToJawSurface()`, procedural fallback tooth meshes, targeted 180-degree flipping, downward tilt correction, surface-clearance nudging, landmark fallback placement, reference-mode switching, or vertex debug overlays.

Viewer Controls

Viewer controls are managed mainly through `src/newControls.js`. On desktop and tablet the controls appear in a slide-in hamburger drawer on the right edge of the viewer. On mobile they appear in a bottom-anchored bar.

Controls Available

- Jaw mesh visibility (upper/lower independently)
- Jaw mesh transparency slider
- Undercut and occlusion heatmap mode
- Polyline component visibility panel
- Artificial teeth jaw visibility (upper/lower independently)
- Approve 3D, Edit 3D, Add to Mail, Show RPD Design action buttons

Some controls call globals registered by `src/index.js` and `src/artificialTeeth.js`:

`window.setPolylineJawVisibility`
`window.setArtificialTeethJawVisibility`

The Objects panel toggle is the `#component-panel-toggle` button, rendered as an icon using `Icon_objects3.png`. The polyline toggle is `#polyline-visibility-toggle`, rendered using `Icon_Hide_SkeletalPrev.png`.

Render Loop

The main render loop runs in `src/index.js`:

```
function animate() {  
  requestAnimationFrame(animate);  
  controls.update();  
  anchorViewerRotationTarget(controls);  
  clampViewerControlTarget(controls);  
  artificialTeethRenderer.syncToJawMeshes();  
  renderer.render(scene, camera);  
}
```

`anchorViewerRotationTarget` and `clampViewerControlTarget` prevent the jaw from drifting out of view during free rotation. `artificialTeethRenderer.syncToJawMeshes()` keeps the artificial teeth overlay groups aligned with their corresponding jaw meshes on every frame.

Loading Screen and Progress

A dedicated loading screen (`#viewer-loading-screen`) is shown over the viewer while data is being fetched. It is structured as a centred card and fades out once the jaw mesh is rendered and the viewer is ready.

The loading card layout from top to bottom:

1. **Icon** — SmartRPD logo
2. **Initialising bar** — animated sweep bar (always visible, indeterminate)
3. **Status text** — current stage label, e.g. "Initialising..." / "Loading jaw..."
4. **Loading bar** — yellow progress bar, visible once a streaming download starts
5. **Percentage and download speed** — shown below the yellow bar during active downloads

The loading elements are accessible globally via `window.viewerLoadingEls` so that `ApiClient` can update the progress bar without importing a UI module. The status text is updated via `window.updateViewerLoading(label)`.

After the viewer becomes interactive, overlay loading (polylines, artificial teeth) continues in the background as `fetchAndRenderCaseOverlays(paramValue)`. This is deliberately fire-and-forget — a failure in overlay loading does not crash the viewer:

```
const overlayLoadPromise = fetchAndRenderCaseOverlays(paramValue).catch(  
  (error) => { console.warn("Case overlays failed to load:", error); }  
);
```

Known Load Failure Fix (Refresh Reliability)

Two bugs caused the viewer to sometimes fail silently on hard refresh:

Bug 1 — ApiClient Content-Length throw (`src/ApiClient.js`): When the GCP proxy returns a response without a **Content-Length** header (common with chunked encoding), the original code threw **'Content-Length response header unavailable'** before reading any data. This caused every API call — case info, mesh data, thumbnails — to fail on affected responses. Fixed by falling back to `response.json()` when **Content-Length** is absent.

Bug 2 — Unguarded positionData access (`src/index.js`): If the case info fetch failed, `positionData` remained **undefined** after the try/catch block. The very next line (`positionData.creation_date`) ran outside the try block and threw an uncaught **TypeError**, aborting the entire viewer IIFE before any meshes were loaded. Fixed by wrapping the metadata display in **if (positionData)** so the viewer continues to load STL and overlay data regardless of whether case info was available.

Performance Strategy

The loading model prioritises making the viewer interactive as early as possible:

1. Login / session establish
2. Fetch case info and thumbnail (parallel)
3. Fetch jaw mesh and denture mesh (parallel, pre-started before case info is awaited)
4. Render jaw mesh → viewer becomes interactive
5. Fetch and render polylines and artificial teeth in the background

The biggest current bottleneck is payload size. Some API responses are between 16 MB and 30 MB. The safe client-side mitigations already in place are:

- Parallel mesh fetches (jaw + denture start simultaneously)
- Progressive loading (viewer is usable before overlays finish)
- Lazy-loaded `chat.js` (off the critical module graph)
- nginx gzip compression for JS/CSS (configured in `nginx.conf`)
- `preconnect` hints for `live.api.smarttpdai.com` in all page heads
- Long-lived cache headers for assets (1 year for JS/CSS, 7 days for images)

The API is served behind a GCP edge (via: `1.1 google, Alt-Svc: h3=":443"`), which means HTTP/3 is available at the edge level. The origin Express server does not need to be upgraded for this benefit to apply to browsers that support it.

Detailed timing diagnostics are available internally via `window.viewerLoadTimings`. They are not logged to the console during normal use. The console flags controlling diagnostic output are at the top of `src/index.js`:

```
const LOG_VIEWER_LOAD_TIMINGS_TO_CONSOLE    = false;
const LOG_VIEWER_OBJECT_COUNTS_TO_CONSOLE   = false;
const LOG_VIEWER_REVISION_TO_CONSOLE         = true;
const LOG_POLYLINE_AUTO_AUDITS_TO_CONSOLE    = false;
const LOG_POLYLINE_LOAD_DETAILS_TO_CONSOLE  = false;
const LOG_ARTIFICIAL_TEETH_STATUS_TO_CONSOLE = false;
```

Suggested Future Backend Improvements

The following backend changes would provide the largest gains beyond what client-side optimisation can achieve:

- Enable gzip or Brotli compression on API responses
- Replace base64-in-JSON with direct binary (MessagePack or multipart) delivery for mesh payloads
- Split large composite endpoints by purpose so the client can request only what it needs
- Add metadata-only endpoints for cache validation (e.g. last-modified timestamp before downloading full mesh)
- Support partial or incremental asset loading

Case List and Chat

The case list page (`src/pages/case_list.html`) is driven by `src/js/caseManagement.js`. It fetches the user's cases from `/case/user/findall/get`, populates a sortable table, and manages case selection, thumbnail preview, co-owner display, and case filtering.

The chat/notes panel (`src/js/chat.js`) is shared between the case list, 3D viewer, and 2D annotation pages. It is lazy-loaded on first button click. On open it fetches historical notes from `/api/smartrpd/notes/get/{caseId}` and begins a 3-second live poll for new messages. The poll pauses when the browser tab is hidden and resumes when the tab regains focus. Messages are threaded by author with right-aligned "You" bubbles and left-aligned bubbles for other users.

Mental Model

Jaw mesh	= anatomical surface – source of truth
Polylines	= design/component overlay layer
Artificial teeth	= separate prosthetic mesh overlay layer
Guidelines	= visual-only reference curves
Controls	= visibility, interaction, and editing affordances
Chat/Notes	= collaborative annotation layer (not 3D, not mesh-related)

The most important rule for artificial teeth:

Geometry source and placement source must match.

The most important rule for polylines:

Coordinate-space detection belongs in the polyline renderer.

The most important rule for loading:

Render the jaw mesh first. Load everything else in the background.

Chunk 1: Overview

Nyunt's work focused mainly on the HTML-based application screens, 2D design workflow and supporting front-end pages around the SmartRPD case lifecycle. This includes the case management page, 2D annotation workspace, shared navigation/sidebar structure, case chat integration, version history access and the clinical/design data flows that support 2D RPD planning.

The HTML application is organised as separate pages rather than a single-page application. Key pages include the case list page for browsing and managing cases, the 2D annotation page for the design workflow, and shared viewer pages such as the 3D viewer. These pages load page-specific CSS and JavaScript modules for case management, case creation, dashboard controls, notifications, version history, viewer shell behaviour, chat, and modular 2D annotation logic.

Chunk 2: Case Management and HTML Workflow

The case management page provides the main operational workspace for selecting and managing cases. It includes case search, status filtering, sortable case tables, thumbnail preview, case detail display, case rename, duplicate and delete actions, 3D viewer launch, copy-link/export actions and a create-case workflow.

The create-case interface supports case metadata entry, upper/lower STL upload, reference image upload, user invitation and progress feedback while backend requests are running.

The page also integrates shared UI elements used elsewhere in the application, including footer status information, app sidebar navigation, notification styling, toast messages, confirmation dialogs and version history access. This gives the application a consistent shell across the case list, 2D annotation and 3D viewer screens.

Chunk 3: 2D Annotation and RPD Design Workflow

The 2D annotation workspace provides the visual design surface for jaw annotation and RPD component planning. The workspace includes upper/lower jaw display, component catalog tabs, tooth-level placement interactions, undo/redo history, save/back handling, clinical information panels, case notes, noticeboard content, chat access and a linked 3D preview area.

The 2D code is separated into focused modules. The main annotation module coordinates page state, case loading, history, save flow and layout behaviour. Other modules handle

the selectable component catalog, component placement, visual rendering, tooth modelling and annotation visuals.

Component-specific modules cover rests, clasps, bars, meshes, plates, minor connectors and major connectors. Jaw-structure modules encode, decode, apply and save the design structure so that 2D design work can round-trip between the interface and backend data.

Chunk 4: Clinical Information and Supporting UI

The 2D workflow also includes clinical information and tooth-status annotation using dedicated clinical assets. Case notes are handled through the case-note workflow, while shared chat is embedded into both the 2D annotation page and 3D viewer shell as a slide-in side panel.

The Version History feature is available from the shared sidebar and is integrated with the version-history page assets. Together with toast messages, confirmation modals and the sidebar component, this supports a more complete application workflow around case review, collaboration and navigation.

Chunk 5: 2D and 3D Integration

The 2D page includes a 3D preview module. This preview supports STL/OFF rendering, jaw visibility controls, upload handling, undercut/heatmap display, camera controls, screen capture and view presets. It connects the 2D planning screen with 3D visual feedback without requiring users to leave the annotation workspace.

In summary, Nyunt's HTML and 2D work provides the application-facing layer for case management and RPD design authoring, while Shafik's 3D work provides the dedicated Three.js viewer and rendering pipeline. The two areas meet through shared case IDs, shared page shell components, chat/version-history integration and common REST API data flows.

Chronological Work Summary

The internship web development work can be read as two connected streams. Nyunt focused on the HTML application screens, case-management flow and 2D design workspace. Shafik focused on the Three.js 3D viewer, 3D rendering architecture and loading/performance improvements. Both streams share the same case data, authentication model, page shell, chat/version-history features and REST API backend.

Application Foundation

The application is a multi-page web app served as static files through nginx, rather than a single-page application.

Main pages:

Login - index.html - Authentication entry point.

Case List - src/pages/case_list.html - Browse, manage, create and open cases.

2D Annotation - src/pages/2DAnnotation.html - 2D jaw annotation and RPD design planning.

3D Viewer - src/pages/ThreeDViewer.html - 3D jaw, design overlay and artificial-teeth visualisation.

Navigation between pages happens through URL redirects. Case identity is passed using an obfuscated encrypted ?id= query parameter, decoded through src/crypt.js. Shared UI modules provide the sidebar, footer, chat panel, notifications, toast messages, confirmation dialogs and version history access.

2D and HTML Workstream: Nyunt

1. Case Management and HTML Page Structure

Nyunt's work begins at the HTML application layer. The case list page provides the main operational workspace for users to browse and manage cases. It includes case search, status filtering, sortable case tables, thumbnail preview, case-detail display, case rename, case duplicate, case delete, 3D viewer launch, copy-link/export actions and the create-case workflow.

The create-case interface supports case metadata entry, upper/lower STL upload,

reference image upload, user invitation and progress feedback while backend requests are running. This makes the case list page the starting point for most user workflows before moving into either 2D design or 3D review.

2. Shared Front-End Shell and Collaboration Features

After the main case-management flow, the HTML pages were connected through shared application UI. The app sidebar, footer status bar, notification styles, toast messages, confirmation dialogs and version-history entry points are reused across the case list, 2D annotation page and 3D viewer shell.

The shared chat/notes panel is available across the case list, 2D annotation page and 3D viewer. It opens as a slide-in side panel, fetches case notes, supports text and image messages, and polls for updates so collaborators can review case-specific discussion without leaving the current workspace.

3. 2D Annotation Workspace

The 2D annotation workspace is implemented through `src/pages/2DAnnotation.html` and the modules under `src/js/2D/`. It provides the visual surface for jaw annotation and RPD component planning. The workspace includes upper/lower jaw display, component catalog tabs, tooth-level placement interactions, undo/redo history, save/back handling, clinical information panels, case notes, noticeboard content, chat access and a linked 3D preview area.

The 2D code is split by responsibility. `2DAnnotation.js` coordinates page state, case loading, history, save flow and layout behaviour. `annotationCatalog.js` manages the selectable component catalog. `annotationPlacement.js`, `annotationRender.js`, `annotationTeethModel.js` and `annotationVisuals.js` handle component placement and visual rendering. Component-specific modules cover rests, clasps, bars, meshes, plates, minor connectors and major connectors.

4. Clinical Information and Jaw-Structure Persistence

The 2D workflow also includes clinical information and tooth-status annotation using dedicated clinical assets and `clinicalInfo.js`. Case notes are handled through the case-note workflow, while jaw-structure modules encode, decode, apply and save the 2D design structure for backend persistence.

This makes the 2D workspace more than a drawing screen: it records structured RPD design information that can be reloaded, edited and round-tripped through the API.

5. 2D-to-3D Preview Link

The 2D page includes a 3D preview module through `preview3D.js`. This preview supports STL/OFF rendering, jaw visibility controls, upload handling, undercut/heatmap display, camera controls, screen capture and view presets. It connects the 2D planning workflow with immediate 3D visual feedback without requiring the user to leave the 2D annotation workspace.

3D Workstream: Shafik

1. Three.js Viewer Foundation

Shafik's work begins with the dedicated 3D viewer architecture in `src/index.js`. The viewer is built with Three.js and uses an orthographic camera so jaw models and design overlays remain at a consistent scale while zooming. The scene contains jaw meshes, design polylines, artificial-teeth overlays, lights, camera controls and the WebGL renderer.

The main scene structure:

THREE.Scene

- parentObject
 - upper/lower jaw mesh objects
 - loaded STL/OFF mesh variants
- polylineOverlayGroup
 - upper/lower component polyline groups
 - tube geometry group
 - edit handle group
- artificial-tooth-overlay-group
 - upper-artificial-teeth group
 - lower-artificial-teeth group
- lights
- camera

2. Jaw Mesh Loading and Surface Rendering

Jaw meshes are loaded from backend endpoints including `/parameterisation/mesh/getall`, `/surface/getall` and `/stl/get`. The viewer supports both OFF and STL mesh formats.

`src/OFFLoader.js` parses OFF data into `THREE.BufferGeometry`, while `src/STLMeshLoader.js` parses STL data, merges vertices and creates mesh objects.

The main anatomical jaw mesh uses `MeshPhongMaterial` with vertex colours. Surface or denture meshes use `MeshStandardMaterial`. Jaw meshes are forced to remain opaque through `enforceOpaqueJawMesh()` so that user transparency controls do not accidentally affect the main anatomical surface.

One important legacy detail is that some upper OFF meshes are rotated and shifted after loading. Because of this, overlay data may arrive either in jaw-local coordinates or already transformed into scene-world coordinates.

3. Polyline Rendering and Coordinate-Space Handling

The polyline system loads design components from `/polylines/getall`, normalises the response, extracts polyline segments, renders tube geometry and creates draggable edit handles. Polyline objects are stored under `polylineOverlayGroup`.

The polyline renderer detects whether each segment is in jaw-local or scene-world coordinates by comparing the segment against jaw vertices in both spaces. If the data is jaw-local, the jaw mesh transform is copied onto the polyline group. If the data is already in scene-world space, the group transform is left unchanged to avoid double-transforming.

This coordinate-space handling belongs in the polyline renderer, not the artificial-teeth renderer. That separation keeps design polylines independent from prosthetic tooth rendering.

4. Artificial Teeth Rendering

Artificial teeth are handled separately in `src/artificialTeeth.js`. They are rendered as a separate overlay layer and do not modify the jaw mesh or polyline system.

The artificial-teeth data flow:

1. `/toothPlacementData/get`
2. `Decode MessagePack / StageDataManager`

3. Map upper/lower tooth craft data
4. Choose geometry source
5. Choose matching placement source
6. Render data
7. Add to artificial-tooth-overlay-group

Each artificial tooth is rendered as a full 3D mesh object with its own geometry, material, position, rotation and transform. The core rule is that geometry source and placement source must match. Surface-mesh geometry uses the API mesh centre as its placement source. Tooth-data geometry uses the matching tooth-data placement fields. Fields [9] and [12] are not used as tooth positions.

Guidelines such as buccal and gingival curves are visual references only. They do not place, snap, constrain, scale or orient artificial teeth.

5. Viewer Controls and Interaction

Viewer controls are managed mainly through `src/newControls.js`. They include upper/lower jaw visibility, jaw transparency, undercut and occlusion heatmap modes, polyline visibility controls, artificial-teeth visibility controls and workflow buttons such as Approve 3D, Edit 3D, Add to Mail and Show RPD Design.

The render loop updates controls, anchors the rotation target, clamps the control target, synchronises artificial-teeth overlay groups to the jaw meshes and renders the scene. This keeps the model centred during rotation and keeps overlay layers aligned with the corresponding jaw meshes.

6. Loading, Reliability and Performance Optimisation

The viewer loading strategy was then improved so the user can interact with the jaw as early as possible.

Current sequence:

1. Establish the shared login/session.
2. Fetch case information and thumbnails.
3. Start jaw mesh and denture mesh requests in parallel.
4. Render the jaw mesh and make the viewer interactive.
5. Load design overlays such as polylines and artificial teeth in the background.

src/ApiClient.js supports streaming progress when a Content-Length header is available. If the API or GCP proxy omits that header, the client falls back to direct JSON parsing instead of failing. This fixes a refresh-reliability issue where missing Content-Length could previously abort API calls before the response body was read.

Another reliability fix guards case metadata display when positionData is unavailable. If case information fails to load, the viewer can still continue loading STL and overlay data instead of throwing an uncaught TypeError.

The main remaining performance limitation is large payload size. Some API responses are approximately 16 MB to 30 MB. Current mitigations include progressive loading, parallel mesh requests, lazy chat loading, gzip-compressed static JS/CSS, long-lived asset cache headers and quieter diagnostic logging.

Integration Summary

Nyunt's 2D and HTML work provides the application-facing layer for case management, clinical annotation and RPD design authoring. Shafik's 3D work provides the dedicated Three.js viewer, mesh rendering pipeline, design-overlay rendering, artificial-teeth visualisation and progressive loading behaviour.

The two workstreams meet through shared case IDs, shared navigation/sidebar components, chat and version-history integration, common REST API endpoints and the 2D-to-3D preview flow. Together, they form the current SmartRPD web application workflow: create or select a case, perform 2D design and clinical annotation, preview or review the design in 3D, collaborate through notes/chat, and continue stabilising the integrated build for UAT and production validation.